Agents Laboratory Report: A Goal-Based Warehouse Agent

Course: CS F407 Artificial Intelligence. **File:** `agents_lab.py` (run with `python3 agents_lab.py`). Numbers below come from running it.

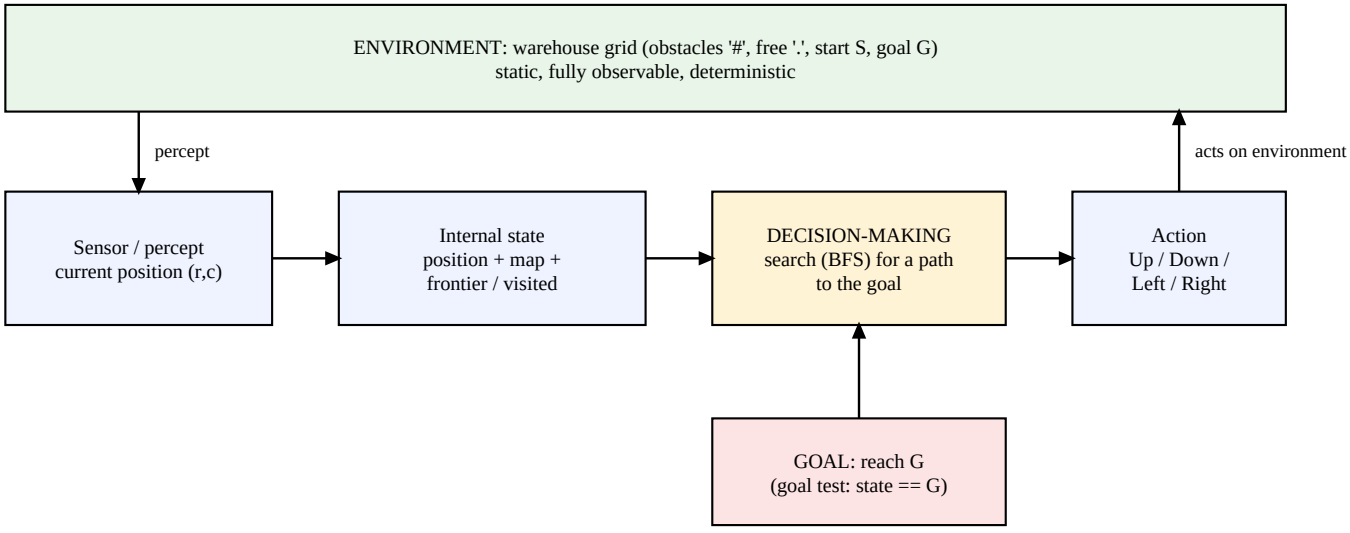
Task 1. Understanding the problem

1. Environment: the 7x21 warehouse grid: '#' obstacles (shelves) that cannot be crossed, '.' free squares, start S and goal G. It is static, fully observable, deterministic, discrete and has a single agent. **2. Goal:** move the vehicle from S to G along a collision-free path. **3. Actions:** Up, Down, Left, Right (one square each). **4. Information the agent maintains:** its current position, the goal position, the map (to know which squares are free), and while searching, a frontier of positions still to be explored and a record of visited positions and how each was reached (to rebuild the path). **5. Why goal-based:** a simple reflex agent maps the current percept straight to an action with condition-action rules and has no explicit objective or look-ahead; such an agent gets stuck in this maze (for example 'move right if free' fails at dead ends). This agent keeps an explicit goal, considers sequences of actions and their consequences, and chooses the sequence that reaches the goal.

Think About It: twice as large. The same strategy stays correct (BFS is complete and finds a shortest path), but the number of states roughly doubles to quadruples, so time and memory grow with it; BFS stores the whole frontier. A heuristic search such as A* would usually be preferable, and other difficulties appear: memory limits, re-planning if shelves move, several vehicles, and partial observability.

Task 2. Design of the agent

Component	Design
Environment	class Environment: grid, start, goal, free(pos), neighbours(pos)
Current state	the vehicle position (row, col); initially S
Goal	the position of G; goal_test(state) returns state == goal
Available actions	Up, Down, Left, Right, valid only if the target square is free
Decision-making	plan(): breadth-first search over positions; returns the action sequence, which act() then executes



Task 3. Prompt engineering and results

[Personalise: replace with the exact prompt(s) you used and your real answers to the four questions]

```
Write a well-documented Python program implementing a goal-based agent for the
warehouse navigation problem shown below [map pasted]. Represent the warehouse as a
2-D grid; find a collision-free path from S to G avoiding obstacles; print the path
(and the map with the path drawn) or a message if no path exists; explain the search
algorithm chosen and why it is appropriate. Use only the standard library. After
writing it, run it and show the output.
```

Result of running the program: a path was found with 20 moves, expanding 59 of the 66 free squares. Path ('*'):

```
####################
#S***.#**********G#
#.##****##########..#
#....##............#
#.######.###.#.###..#
#........#.........#
####################
```

Move sequence: Right Right Right Down Right Right Right Up Right Right Right Right Right Right Right Right Right Right Right Right

Failure test: on the map `#####/#S#G#/#####` the agent reports no path (found = False) and terminates, because visited states are never re-queued.

Check: the final position equals G (an assertion in `run()`), the path never enters '#', and each step moves exactly one square; the 20-move length equals the shortest route (BFS guarantee for unit costs).

1. Working on the first attempt? Programs for this standard problem usually run, but this must be tested rather than assumed: run it, and test an unreachable goal. **2. Improving the prompt:** name the algorithm and state representation, require the standard library only, require the path to be drawn on the map, require a clear 'no path' message, and give a failing test case. **3. Search algorithm:** breadth-first search (A* with Manhattan distance is the common alternative). **4. Why:** every move costs the same, so BFS is complete and returns a shortest path, it is simple, and the grid is small enough for its memory use to be acceptable.

[Personalise: answers 1 and 4 should reflect which algorithm and behaviour YOUR LLM produced]